Week 10 Level 3: Advanced

Project Studio 1

Design first: classes, data and milestones – then build the core.

What you will do

- Design your project on paper with **CRC cards** and a **class diagram**.
- Plan your **milestones**: small steps you can finish and save.
- Start a **git repository** for your project and make your first commits.
- Check that every picture, sound or library you use is **allowed**.

Design first

Builders draw plans before they pour concrete. Programmers design before they type. Your design answers three questions: *Which classes do I need? What does each one know and do? Who works with whom?*

CRC cards

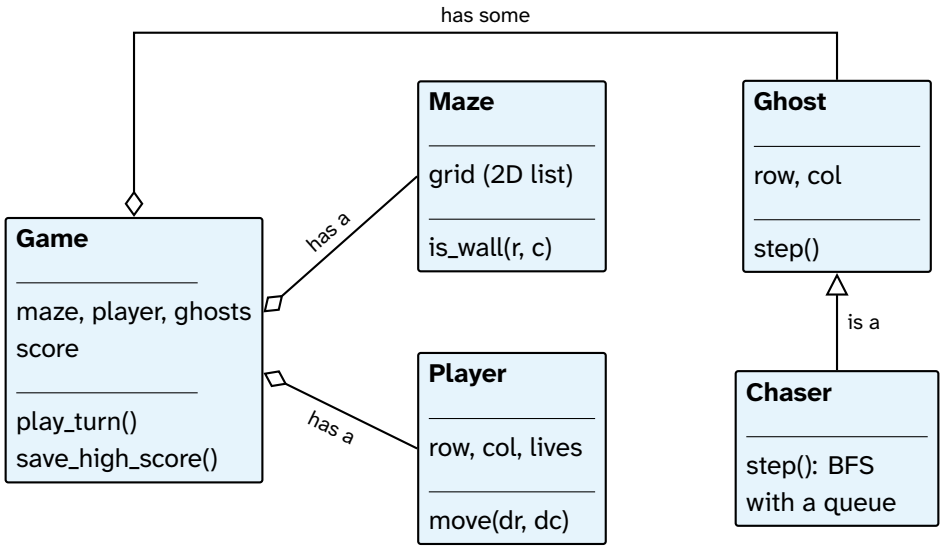
A **CRC card** is an index card for one class: the **C**lass name, its **R**esponsibilities (what it knows and does) and its **C**ollaborators (the classes it works with). Here is one from the reference project, *Maze Muncher*:

Class: Chaser (is a Ghost)	
Responsibilities – knows its row and column (from Ghost) – each turn, moves one step towards the player – finds the shortest path with breadth-first search and a queue	**Collaborators** Maze (where are the walls?) Player (where is it?)

Tip: if a card's responsibility list gets long, that class is doing too much – split it in two.

Class diagrams: “is a” and “has a”

A class diagram shows all the classes and how they connect. An arrow with a hollow triangle means **is a** (inheritance). A line with a diamond means **has a** (composition).



Read it aloud: *a Game has a Maze, a Player and some Ghosts; a Chaser is a Ghost.*

Milestones and save points

Big projects are built in small steps. After each step works, you **commit** – a save point you can always return to.



A good commit message starts with a verb and says what changed: `Add Maze class that loads a grid`, `Fix player walking through walls`. Your project needs **8 or more** meaningful commits by the showcase.

Exercise 10.1: My CRC cards

Write one card per class (at least two classes; one must use inheritance or composition). Copy them here.

Class	**Responsibilities** (knows / does)	**Collaborators**

Exercise 10.2: My class diagram

Draw a box for each class (name, what it knows, what it does). Join them with “is a” arrows and “has a” lines. Then write one sentence: … *is a* … or … *has a* ….

Exercise 10.3: My milestone plan

Milestone	**What will work**	**Done? (date)**
M1 Pitch	My idea and pitch card (week 9)	
M2 Core	My classes exist and the smallest version runs (week 10)	
M3 Features	+ 5 tests + a playtest (week 11)	
M4 Polish	+ README + accessibility fixes (week 11)	
Showcase	2-minute talk (week 12)	

Exercise 10.4: My repository and first commits

- ☐ I copied the starter folder to `Documents/coding-projects` and renamed it.
- ☐ I ran `git init` inside my project folder (I checked with pwd first).
- ☐ Each class has an `__init__` and empty methods, and the file runs without errors.
- ☐ I have at least 2 commits. My best message: _____

Digital citizenship & safety: May I use this? Licenses

Everything in your project must be yours or allowed. Python's standard library is free to use, and your own drawings and sounds are yours. For anything made by someone else, look at its **license**: **CC0** = free for anyone; **CC BY** = free if you give credit; "All rights reserved" or no license = **do not use**. Never download anything without a parent, and list every borrowed item in the "Credits" part of your README.

Vocabulary box

CRC card A card for one class: Class name, Responsibilities, Collaborators.

class diagram A drawing of classes as boxes, joined by "is a" and "has a" links.

composition "Has a": an object is built from other objects (a Game has a Maze).

milestone A small goal on the way to the finished project.

repository A project folder that git keeps a history of.

license The rules that say how you may use someone's work.

Reflection

Which class was hardest to describe? Why?

What will your next commit message be?

Week 10 Level 3: Advanced

Project Studio 1

15–20 minutes of project work. Do Core first; Challenge and Creative are optional.

Name (nickname is fine): _____ Date: _____

★ Core (everyone): Build your core (Milestone 2)

Work on your project in `Documents/coding-projects/<your project>`.

1. Pick **one** class from your CRC cards and make its `__init__` and one method really work.
2. Run it. When it works, commit in Terminal:

>_ Terminal

```
git add .
git commit -m "Make Player move one cell"
```

3. Repeat for another small step. Aim for **3 or more new commits** this week.

Copy your new commit messages from `git log --oneline`:

★★ Challenge (want more?): Your first test

Create `test_<your project>.py` next to your code and write one test for the method you just finished (look at the example in workbook chapter 11). Run it in Thonny with **Tools** → **Open system shell...** and `python -m pytest -q`. Does it pass? Break the code on purpose: does it fail?

Creative (make it yours): Title screen

Design the first thing a player sees: the name of your project, the controls, and a picture made with turtle. Use the Okabe–Ito palette and write the controls in text (not color alone). Sketch it here, then code it if you have time.

Self-check: tick each box when it is true

- ☐ My project folder is a git repository (`git log` works there).
- ☐ At least one class really works, not just `pass`.
- ☐ I made at least 3 commits with messages that start with a verb.
- ☐ Everything in my project is mine or allowed by its license, and listed for my Credits.